CSE 101

Algorithm Design and Analysis

Qipeng Liu

Russell Impagliazzo

Kruskal's algorithm

DSDS

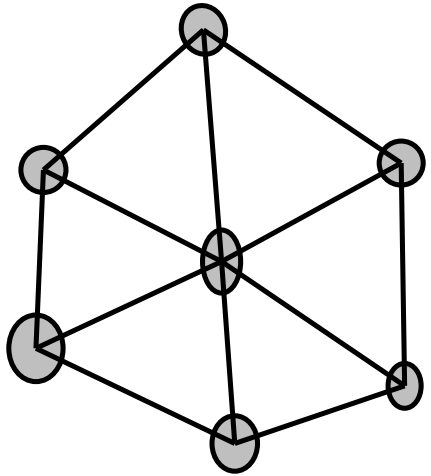
Today’s plan

Kruskal’s algorithm

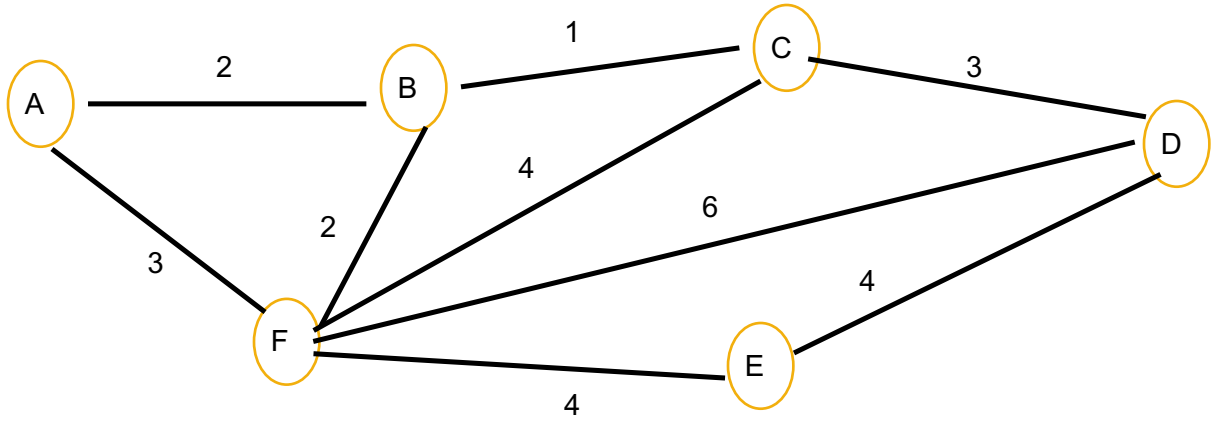
Data structures for disjoint sets

Spanning trees

- A spanning tree of an undirected graph G=(V,E) is a subgraph G'=(V,E') such that
- G' is a tree and
- all vertices in V are connected.

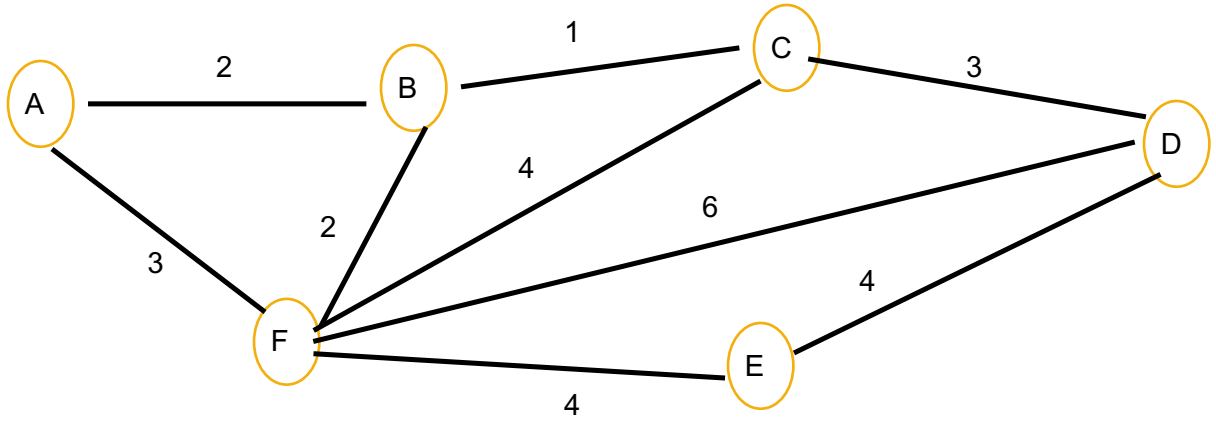


EXAMPLE



What is a minimum spanning tree of this graph?

EXAMPLE

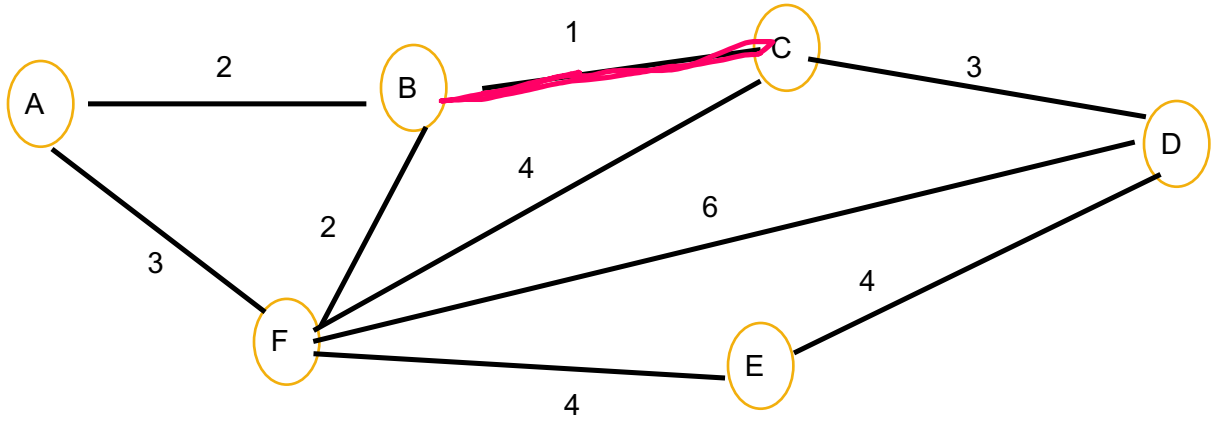


Greedy rule: which edge should we pick first?

Two greedy algorithms:

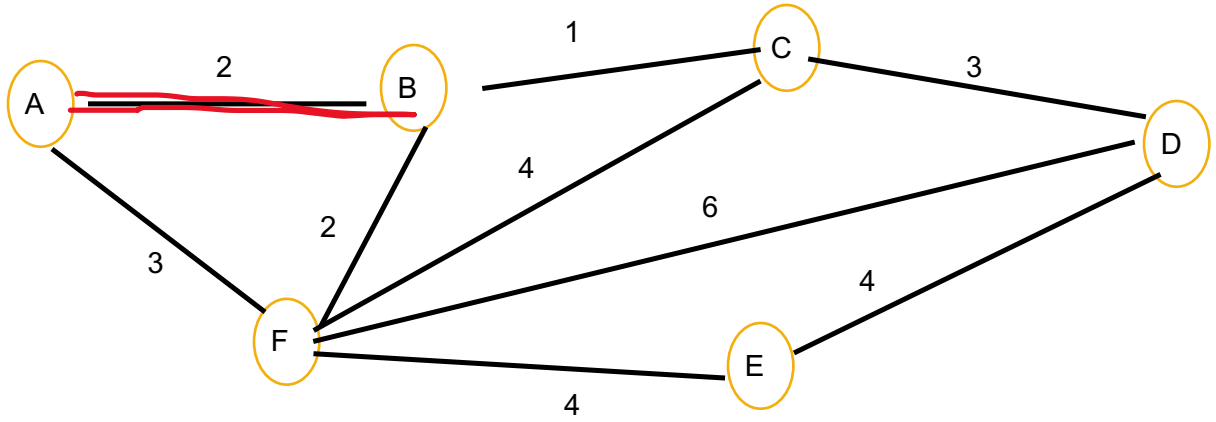
- Kruskal's: Pick the lightest edge. Then repeatedly pick the next lightest edge that doesn't create a cycle
- Prim's: Pick the lightest edge incident to s. Then repeatedly pick the next lightest edge that expands the tree to a new vertex

EXAMPLE: Kruskal



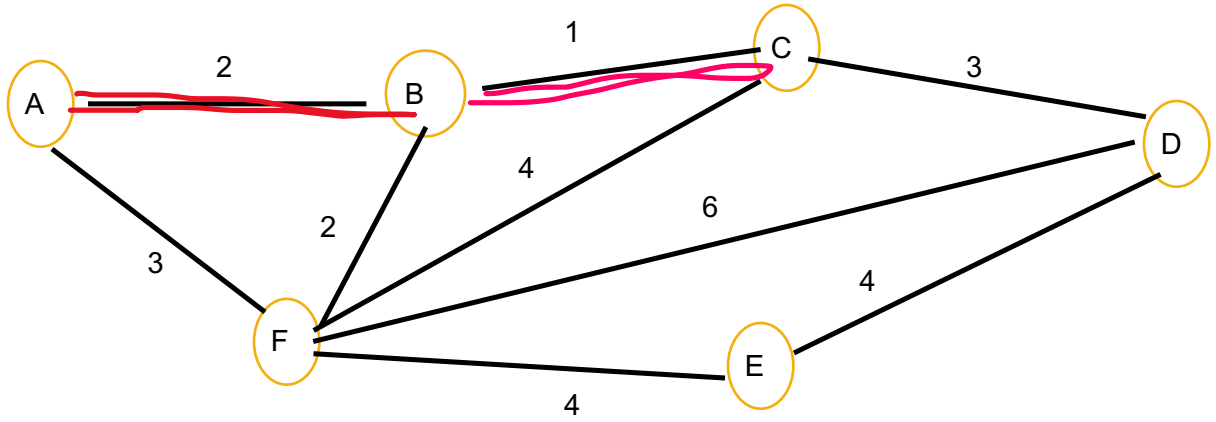
Greedy rule: which edge should we pick first?

EXAMPLE: Prim



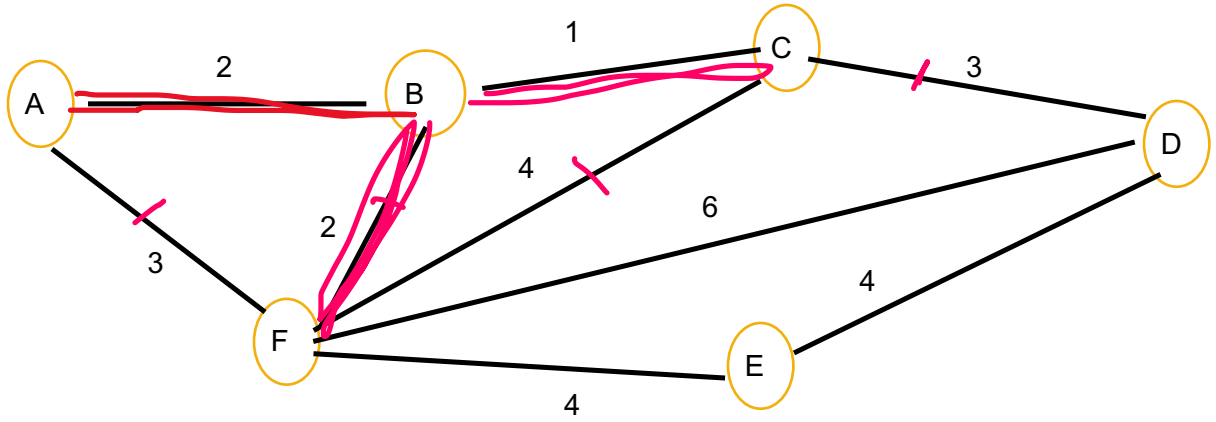
Greedy rule: which edge should we pick first?

EXAMPLE: Prim



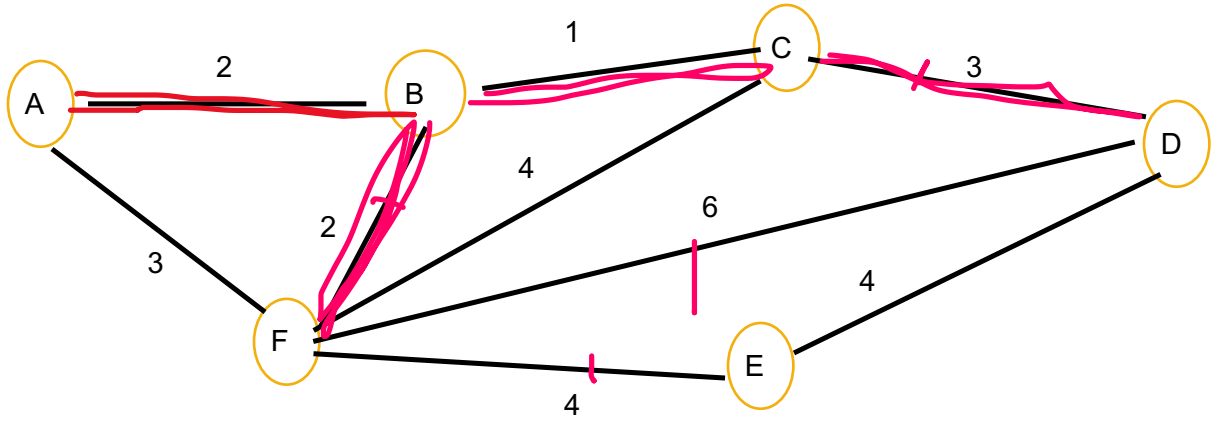
Greedy rule: which edge should we pick first?

EXAMPLE: Prim



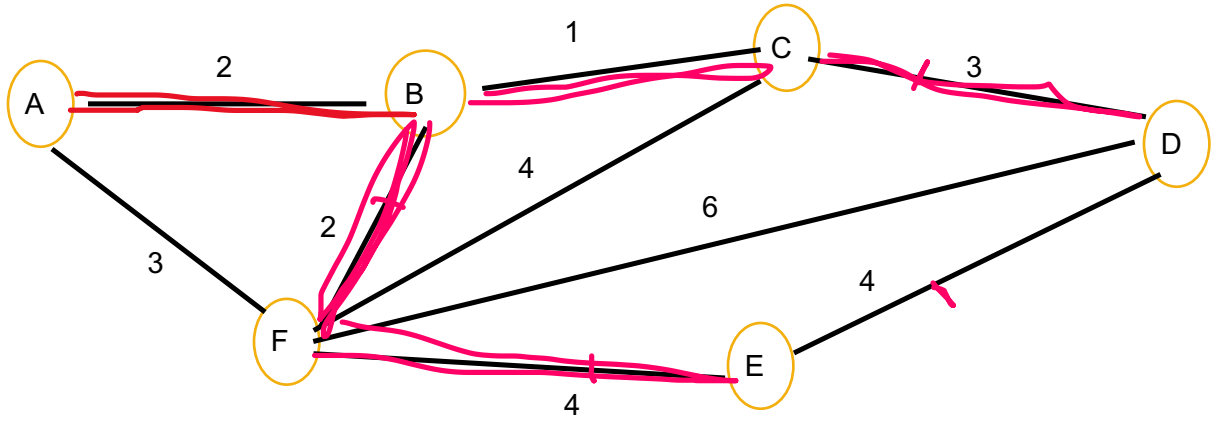
Greedy rule: which edge should we pick first?

EXAMPLE: Prim



Greedy rule: which edge should we pick first?

EXAMPLE: Prim



Greedy rule: which edge should we pick first?

Correctness for Prim's:

- Prim's: Keep a tree rooted at s, starting with the empty tree. Repeatedly pick the next lightest edge that expands the tree to a new vertex.
- P_k the tree with k+1 vertices and k edges rooted at s after k steps
- Lemma: For all $0 \leq k \leq n$, there is a MST T_k on the whole graph with $P_k \subseteq T_k$

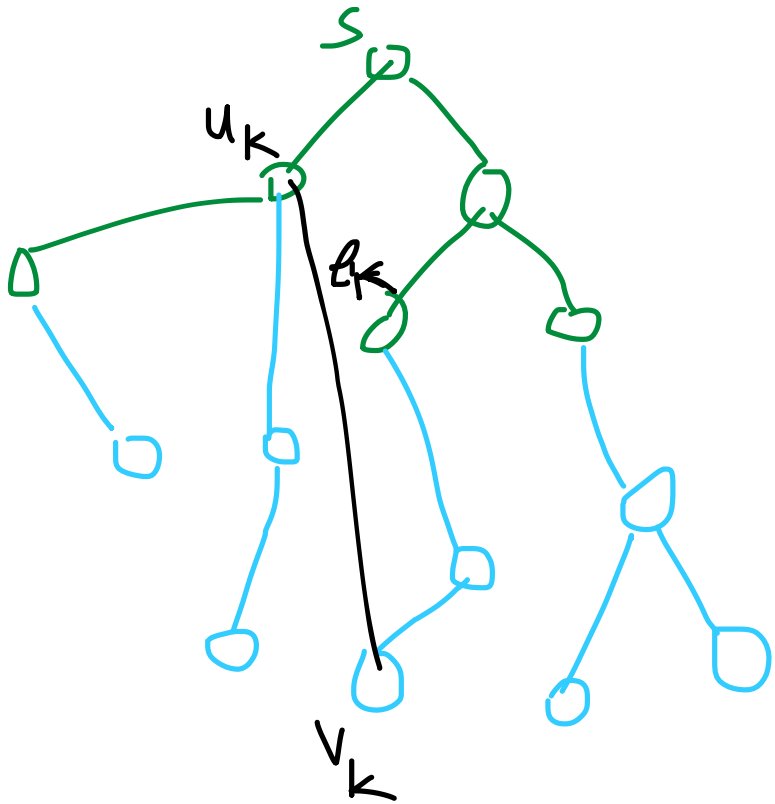
Correctness for Prim's:

- Lemma: For all $0 \leq k \leq n$, there is a MST T_k on the whole graph with $P_k \subseteq T_k$
- Base case: k=0. Trivial. Let T_0 be any MST,
- Since P_0 is empty, $P_0 \subseteq T_0$

Correctness for Prim's:

- Lemma: For all $0 \leq k \leq n$, there is a MST T_k on the whole graph with $P_k \subseteq T_k$
- Induction step: Assume $P_{k-1} \subseteq T_{k-1}$
- Let $e_k = \{u_k, v_k\}$ be the edge Prim's algorithm adds, with $u_k \in P_{k-1}$, $v_k \notin P_{k-1}$.

Correctness for Prim's:

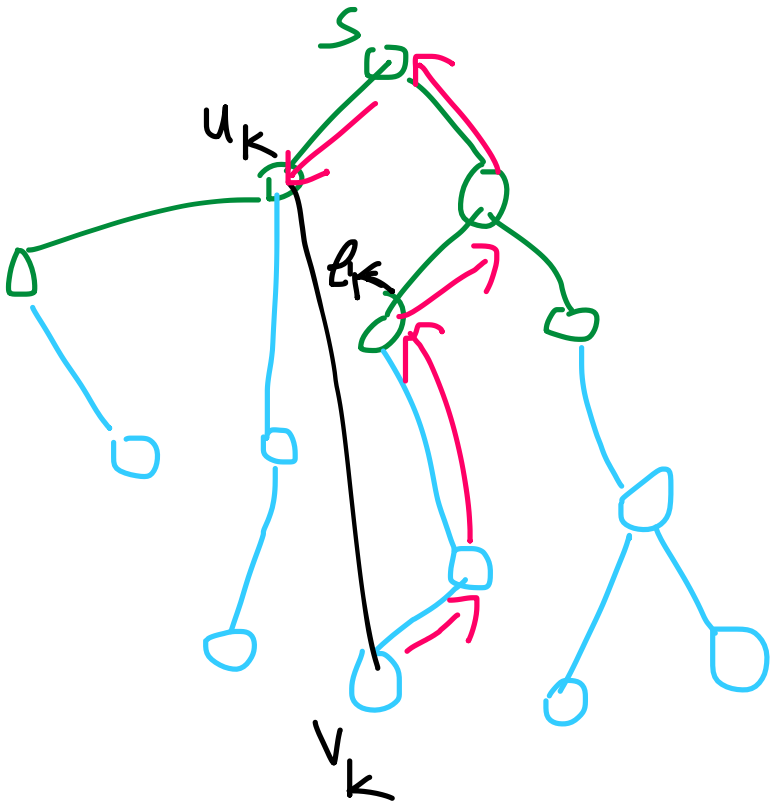


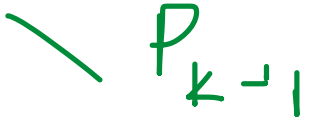
P_{k-1}

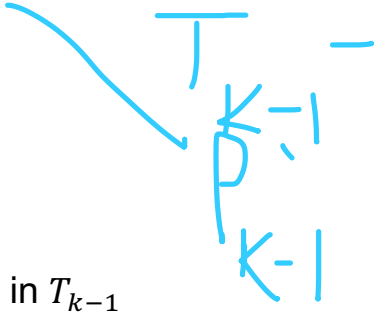
T_{k-1}

P_{k-1}

Correctness for Prim's:

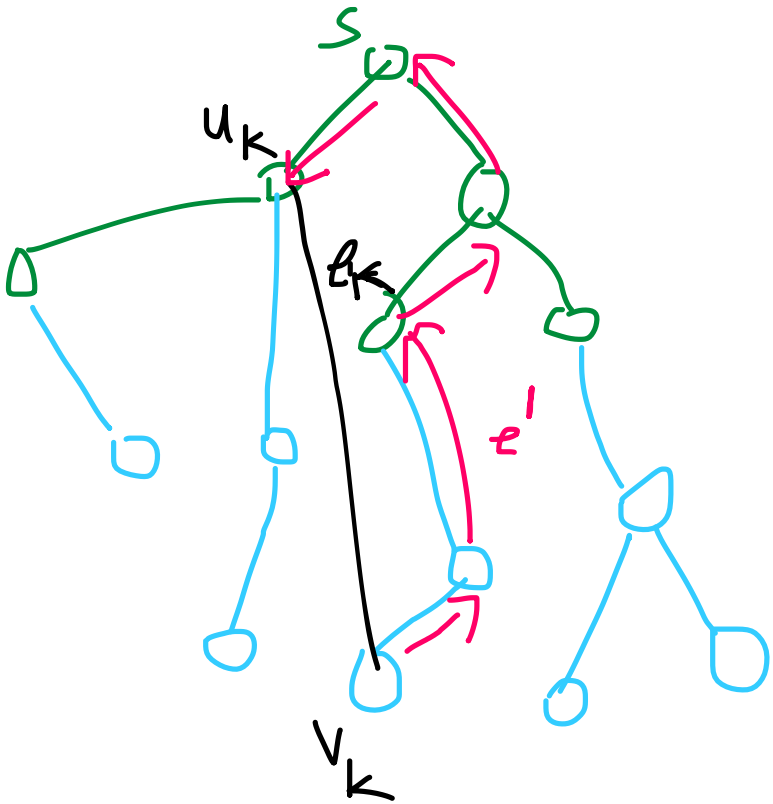


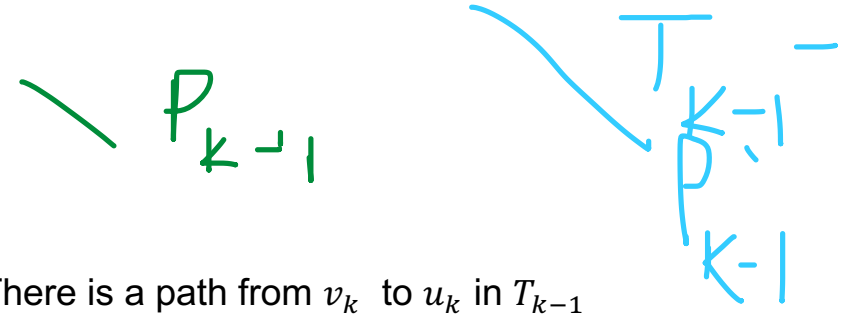




There is a path from v_k to u_k in T_{k-1}

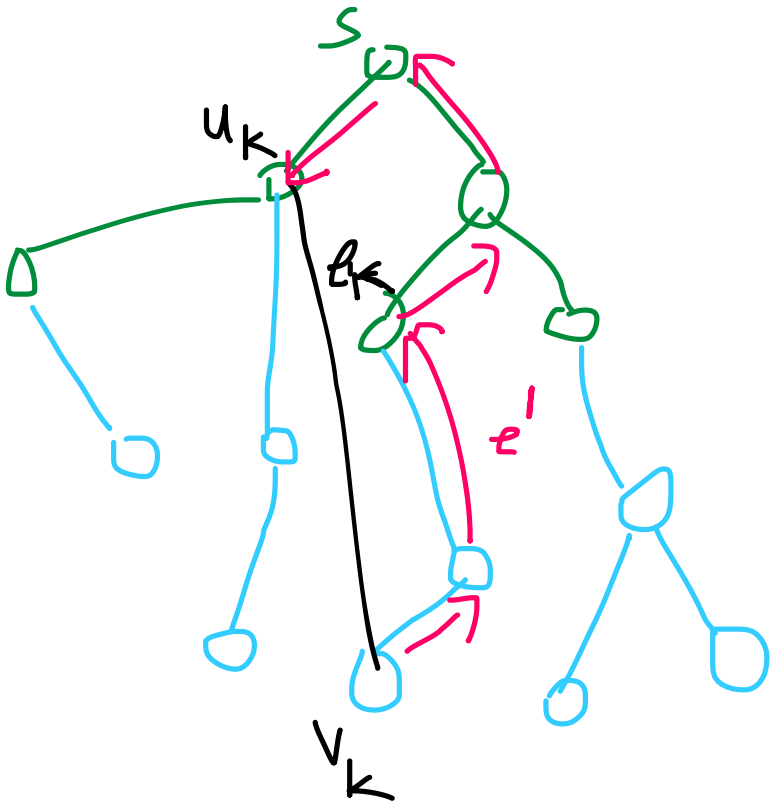
Correctness for Prim's:

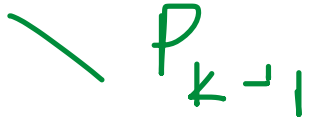


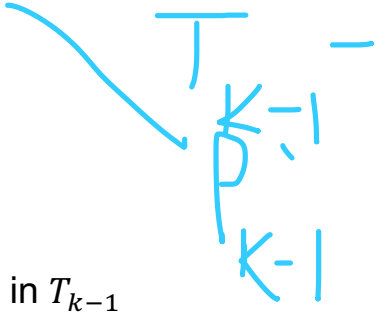


There is a path from v_k to u_k in T_{k-1}
Along that path, some edge e' must go from outside P_{k-1} to inside it.

Correctness for Prim's:



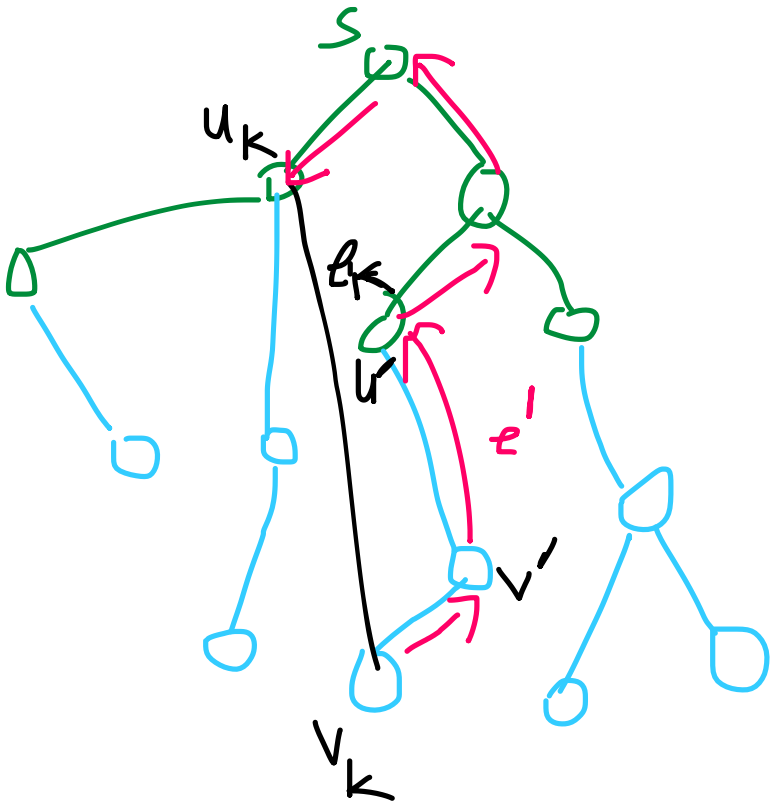


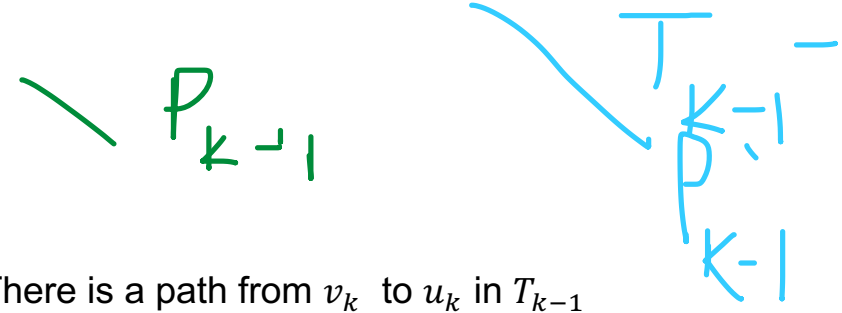


There is a path from v_k to u_k in T_{k-1}
Along that path, some edge e' must go from outside P_{k-1} to inside it.

Let $T_k = T_{k-1} \cup \{e_k\} - \{e'\}$

Correctness for Prim's:



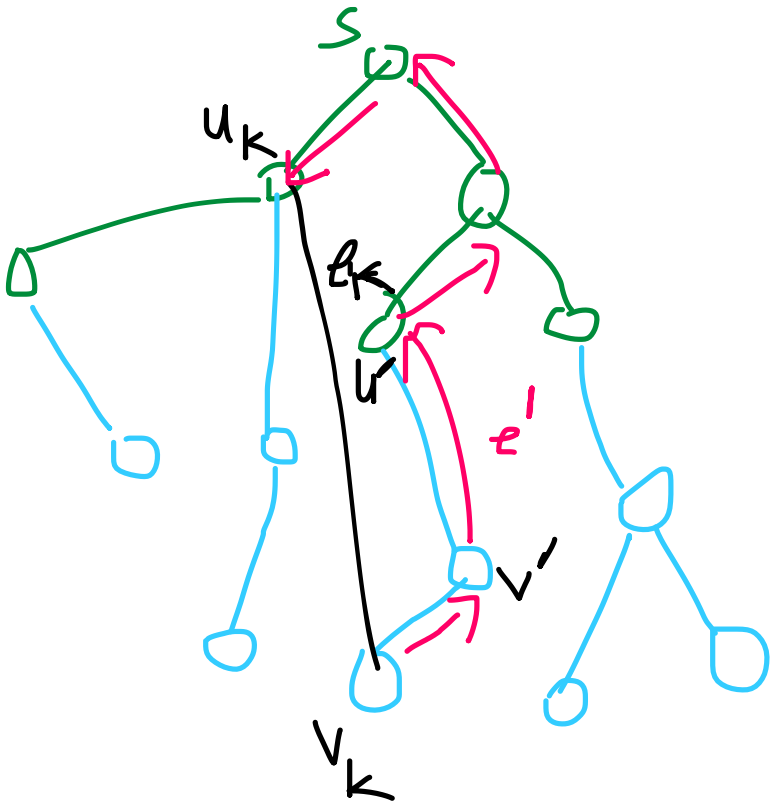


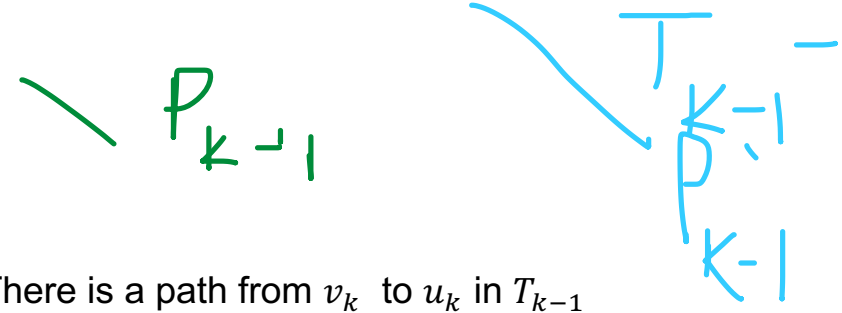
There is a path from v_k to u_k in T_{k-1}
Along that path, some edge e'={u',v'} must go from outside P_{k-1} to inside it.

Let $T_k = T_{k-1} \cup \{e_k\} - \{e'\}$

T_k is still connected, because we can go from u' to u_k , use e_k to get to v_k and then go from v_k to v'
So T_k is a spanning tree.

Correctness for Prim's:





There is a path from v_k to u_k in T_{k-1}
Along that path, some edge e'={u',v'} must go from outside P_{k-1} to inside it.

$w(e_k) \leq w(e')$ or else Prim would have chosen e'.

Let $T_k = T_{k-1} \cup \{e_k\} - \{e'\}$

$$Cost(T_k) = Cost(T_{k-1}) + w(e_k) - w(e') \leq Cost(T_{k-1})$$

Implementing Prim's

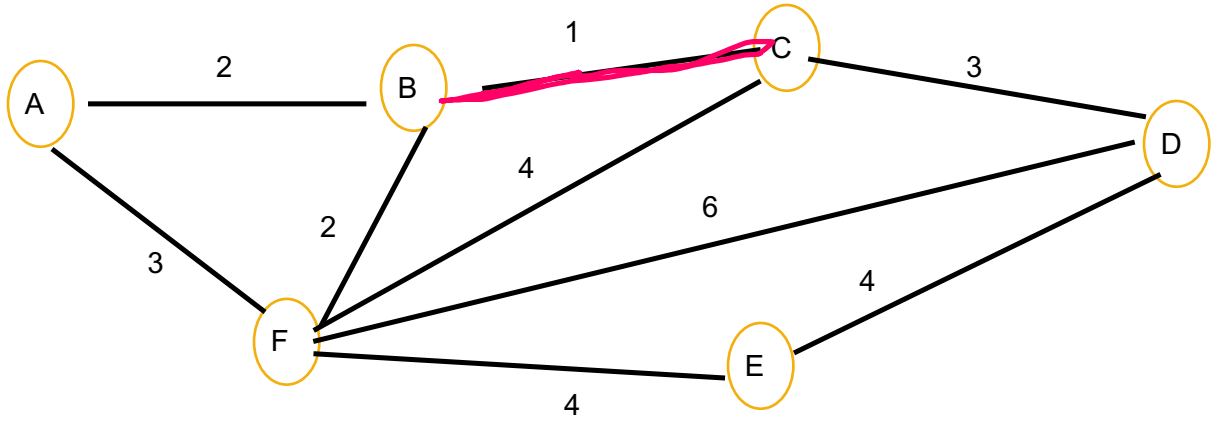
Let F be the vertices not yet in the forest, X the vertices in the forest.

We need to keep track of, for u in F, the minimum weight edge from some v in X to u.

We find the minimum overall such weight, add the edge, and move the u from F to X. We possibly decrease keys for neighbors of u, if the weight from u is smaller.

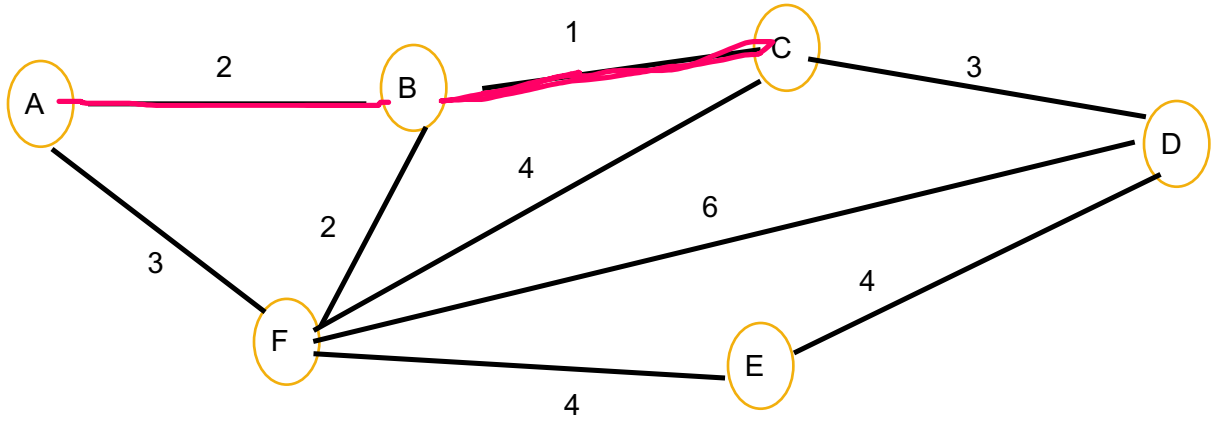
This makes Prim's algorithm almost identical to Dijkstra's for implementation.

EXAMPLE: Kruskal



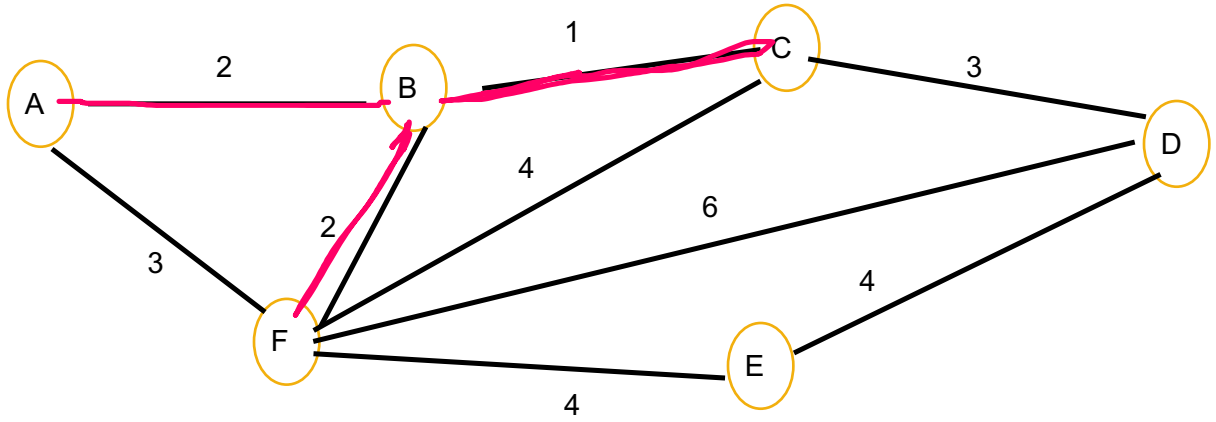
Greedy rule: which edge should we pick first?

EXAMPLE: Kruskal



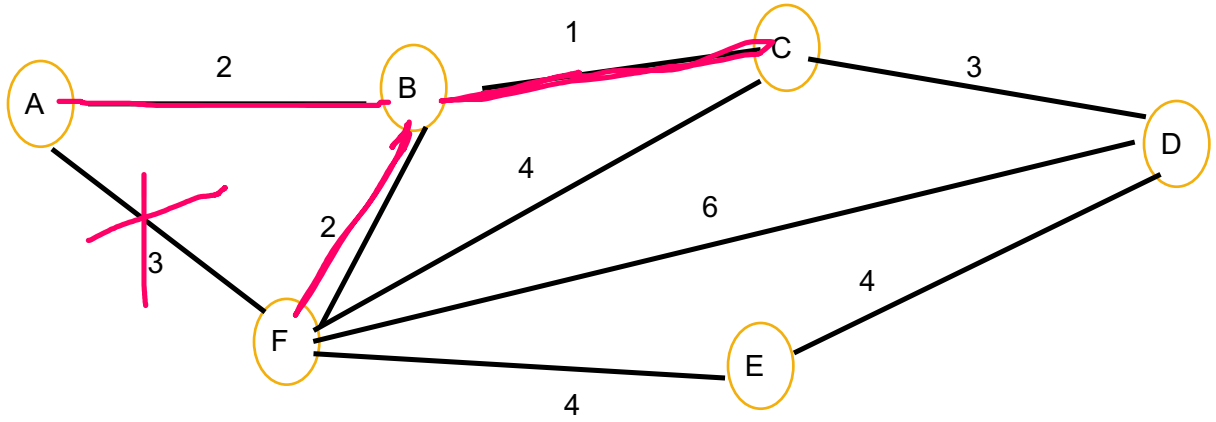
Greedy rule: which edge should we pick first?

EXAMPLE: Kruskal



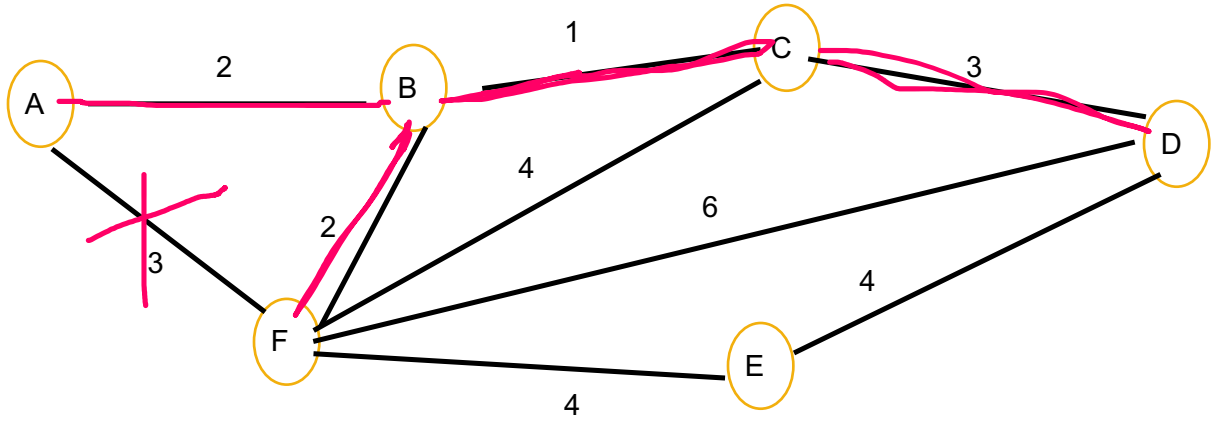
Greedy rule: which edge should we pick first?

EXAMPLE: Kruskal



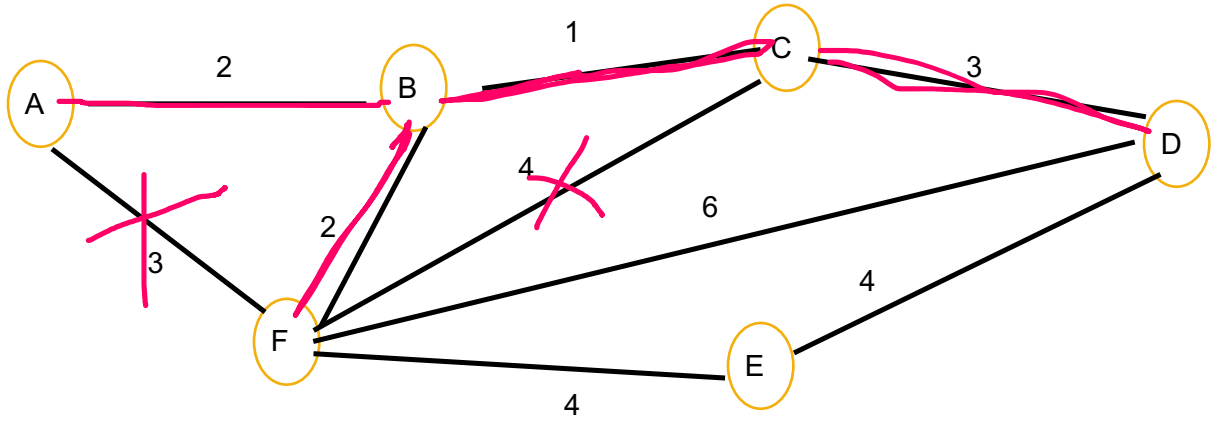
Greedy rule: which edge should we pick first?

EXAMPLE: Kruskal



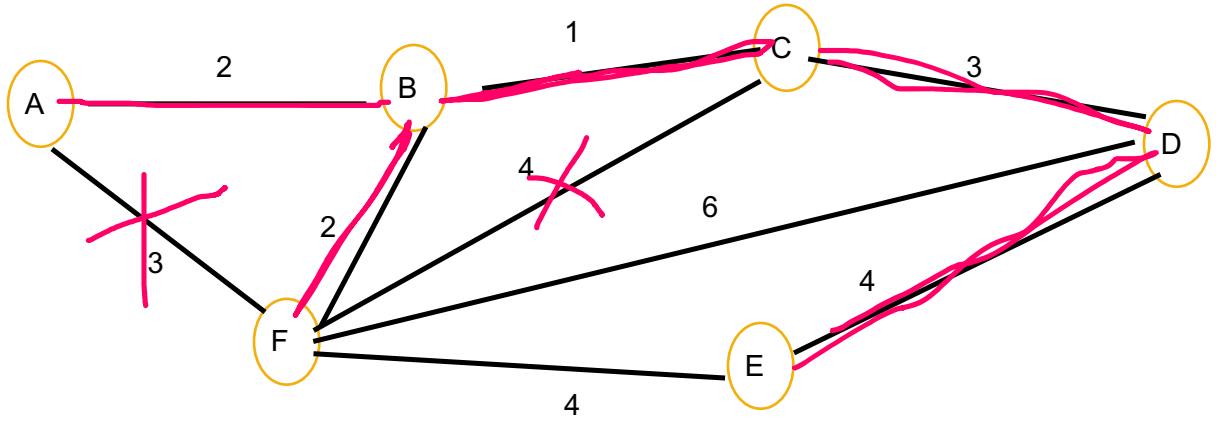
Greedy rule: which edge should we pick first?

EXAMPLE: Kruskal



Greedy rule: which edge should we pick first?

EXAMPLE: Kruskal



Greedy rule: which edge should we pick first?

Kruskal's correctness

- Two steps:
- 1. a helpful loop invariant, characterizing what Kruskal's algorithm does. This will also be useful in understanding what DS to use for Kruskal.
- 2. A greedy-stays-ahead proof using the invariant as a lemma.

Kruskal's as iterative algorithm

- Kruskal (G)
- 1. Sort the edges from smallest weight to largest weight, as $e_1, \ldots . e_m$. Initialize an empty forest T
- 2. For each edge e_i in order do:
- If $T \cup \{e_i\}$ doesn't have a cycle
- Then $T = T \cup \{e_i\}$

Kruskal's loop invariant

1. Sort the edges from smallest weight to largest weight, as $e_1, \ldots . e_m$. Initialize an empty forest T

- 2. For each edge e_i in order do:
- If $T \cup \{e_i\}$ doesn't have a cycle
- Then $T = T \cup \{e_i\}$

Let $G_i = (V, \{e_1, \ldots e_i\}), T_i \; be \; T \; after \; looking \; at \; e_1 \; \ldots e_i$

Kruskal's loop invariant

1. Sort the edges as $e_1, \ldots . e_m$.
2. For each edge e_i in order do:
   If $T \cup \{e_i\}$ is a forest Then $T = T \cup \{e_i\}$

Let $G_i = (V, \{e_1, \ldots e_i\}), T_i \; be \; T \; after \; looking \; at \; e_1 \; \ldots e_i$

Invariant: The connected components in G_i are the same as in T_i

Kruskal's loop invariant

1. Sort the edges as $e_1, \ldots . e_m$.
2. For each edge e_i in order do:
   If $T \cup \{e_i\}$ is a forest Then $T = T \cup \{e_i\}$

Let $G_i = (V, \{e_1, \ldots e_i\}), T_i$ *be* T *after looking at* $e_1 \ldots e_i$

Invariant: The connected components in G_i are the same as in T_i

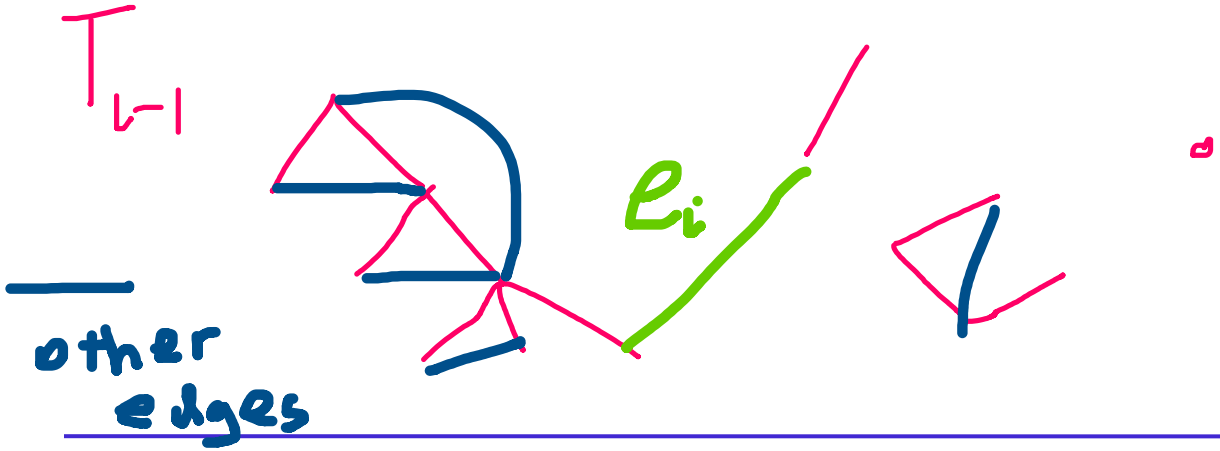
Base case: i=0, both are empty graphs, components are both each individual vertex

Kruskal's loop invariant

1. Sort the edges as $e_1, \ldots . e_m$.
2. For each edge e_i in order do:
   - If $T \cup \{e_i\}$ is a forest Then $T = T \cup \{e_i\}$

Let $G_i = (V, \{e_1, \ldots e_i\}), T_i \; be \; T \; after \; looking \; at \; e_1 \; \ldots e_i$

Induction step: Assume the connected components in G_{i-1} are the same as in T_{i-1}. Case 1: $e_i \; gets \; added$



e_i gets added if it connects two different components in T_{i-1}. Since the same two components get merged in G_{i-1},
the two components are merged together in both T_i and G_i.

So components are the same after e_i. is added

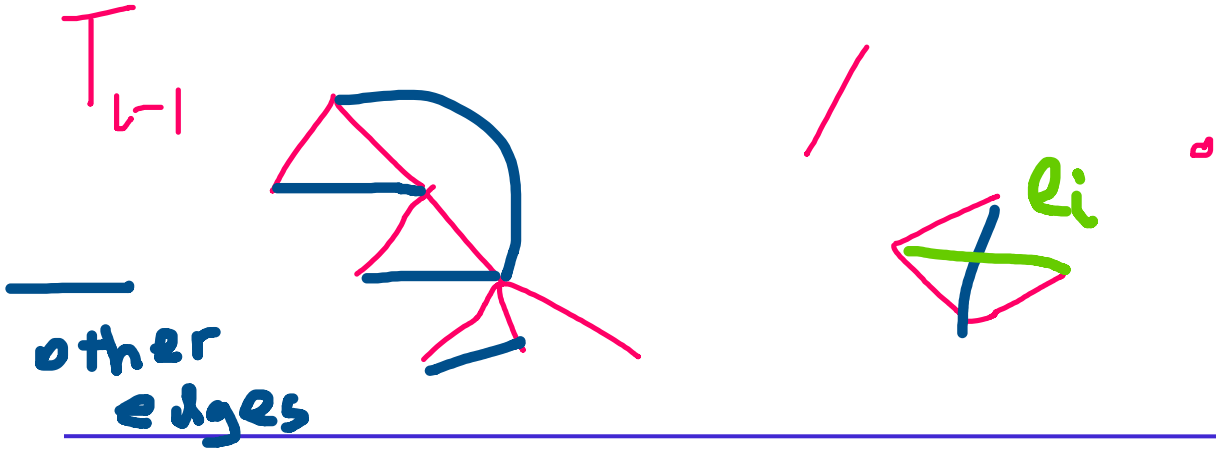
Kruskal's loop invariant

1. Sort the edges as $e_1, \ldots . e_m$.
2. For each edge e_i in order do:
   If $T \cup \{e_i\}$ is a forest Then $T = T \cup \{e_i\}$

Let $G_i = (V, \{e_1, \ldots e_i\}), T_i \; be \; T \; after \; looking \; at \; e_1 \; \ldots e_i$

Induction step: Assume the connected components in G_{i-1} are the same as in T_{i-1}. Case 2:

$e_i \; does \; not \; get \; added$



e_i doesn't gets added if it connects vertices in the same component in T_{i-1}. Since they are also in the same component in G_{i-1}, the components don't change in either T_i or G_i.

So components are the same after e_i is added to G_i

Greedy stays ahead proof.

Both OT and Kruskal are spanning trees, so can't contain cycles. So after adding J edges, they will have n-J components.

OT in order : $e_{i_1}, e_{i_2}, \quad \ldots\ldots\ldots e_{i_{n-1}}$

Kruskal in order: $e_1, e_{K_2}, \ldots . e_{K_{n-1}}$

Greedy stays ahead proof.

Both OT and Kruskal are spanning trees, so can't contain cycles. So after adding J edges, they will have n-J components.

OT in order : $e_{i_1}, e_{i_2}, \quad \ldots\ldots\ldots e_{i_{n-1}}$

Kruskal in order: $e_1, e_{K_2}, \ldots . e_{K_{n-1}}$

Claim: For all j, $i_j \geq K_j$. This implies $w\left(e_{i_j}\right) \geq w\left(e_{K_j}\right)$
Proof: if $i_j < K_j$, we have n-j components using only edges from G_{K_j-1}.
But $T_{\{K_J-1\}}$ has n-j+1 components, since we've only added j-1 edges,
and the invariant says the two sets of components are the same.
From this contradiction, $i_j \geq K_j$

Making Kruskal's algorithm efficient

How to implement Kruskal's

- Sort edges by weight, go through from smallest to largest, and add if it does not create cycle with previously added edges
- How do we tell if adding an edge will create a cycle?

How to implement Kruskal's

- Sort edges by weight, go through from smallest to largest, and add if it does not create cycle with previously added edges
- How do we tell if adding an edge will create a cycle?
- Naive : DFS every time
- Need to test for every edge, m times
- DFS on a forest: only edges added to MST searched
- Thus, each DFS is O(|V|).
- Total time O(|V||E|)

Data structures for Kruskal's

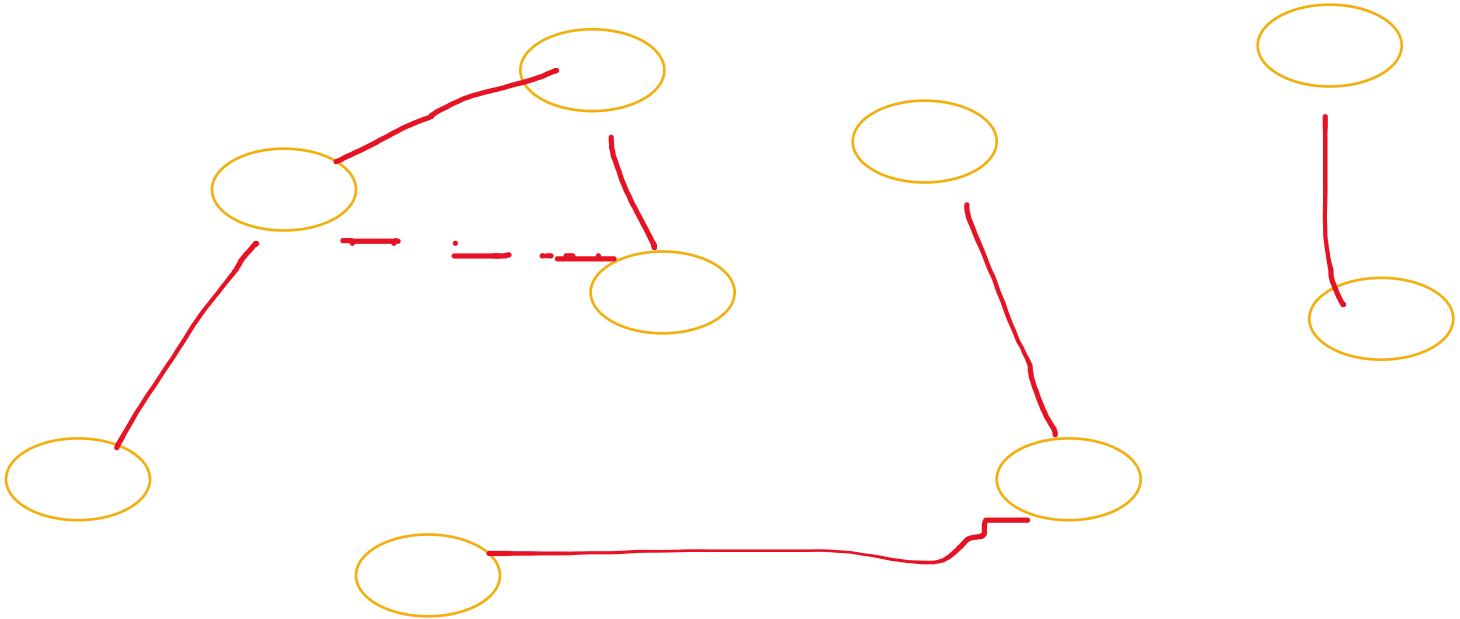
- What kind of information do we need to keep track of in Kruskal's

algorithm?

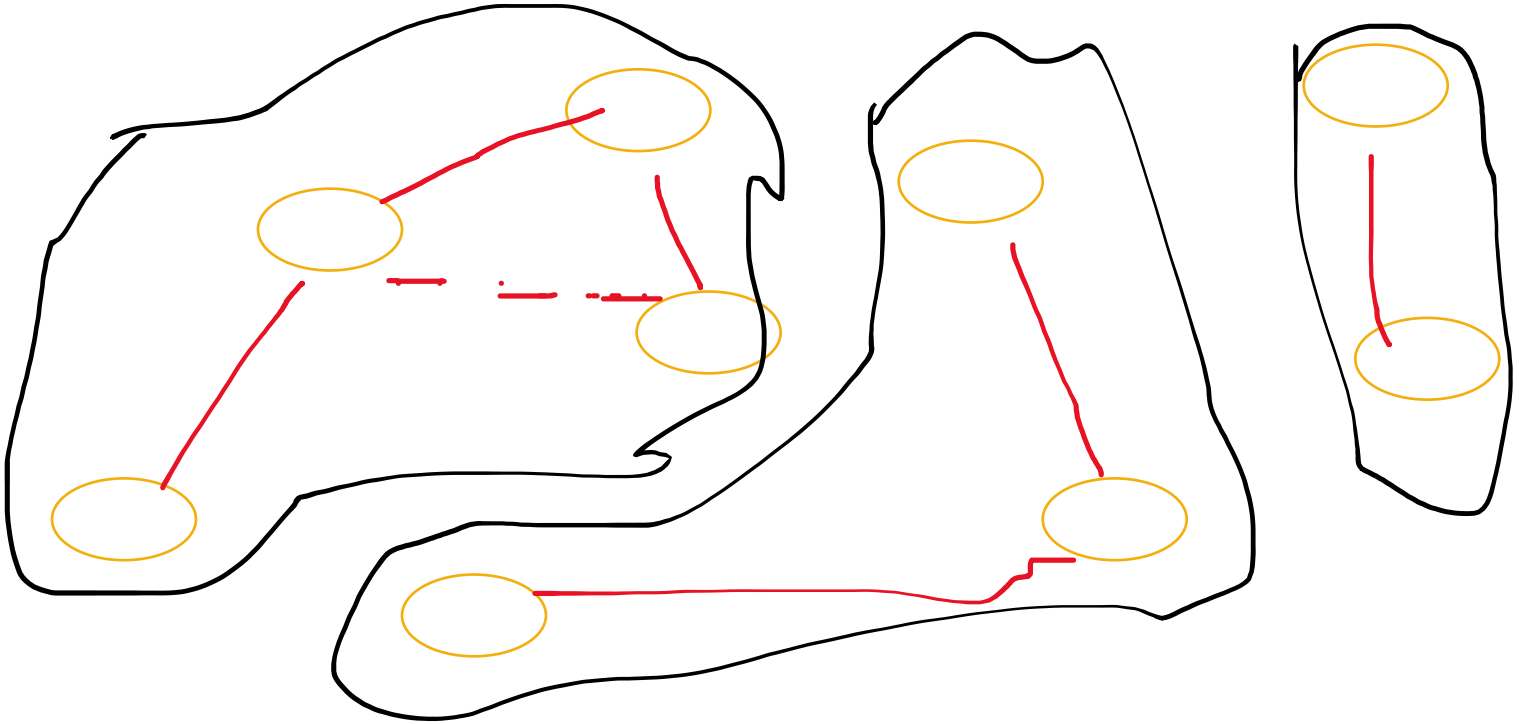
How do we need to access this information in one step?

How does this information change in one step?

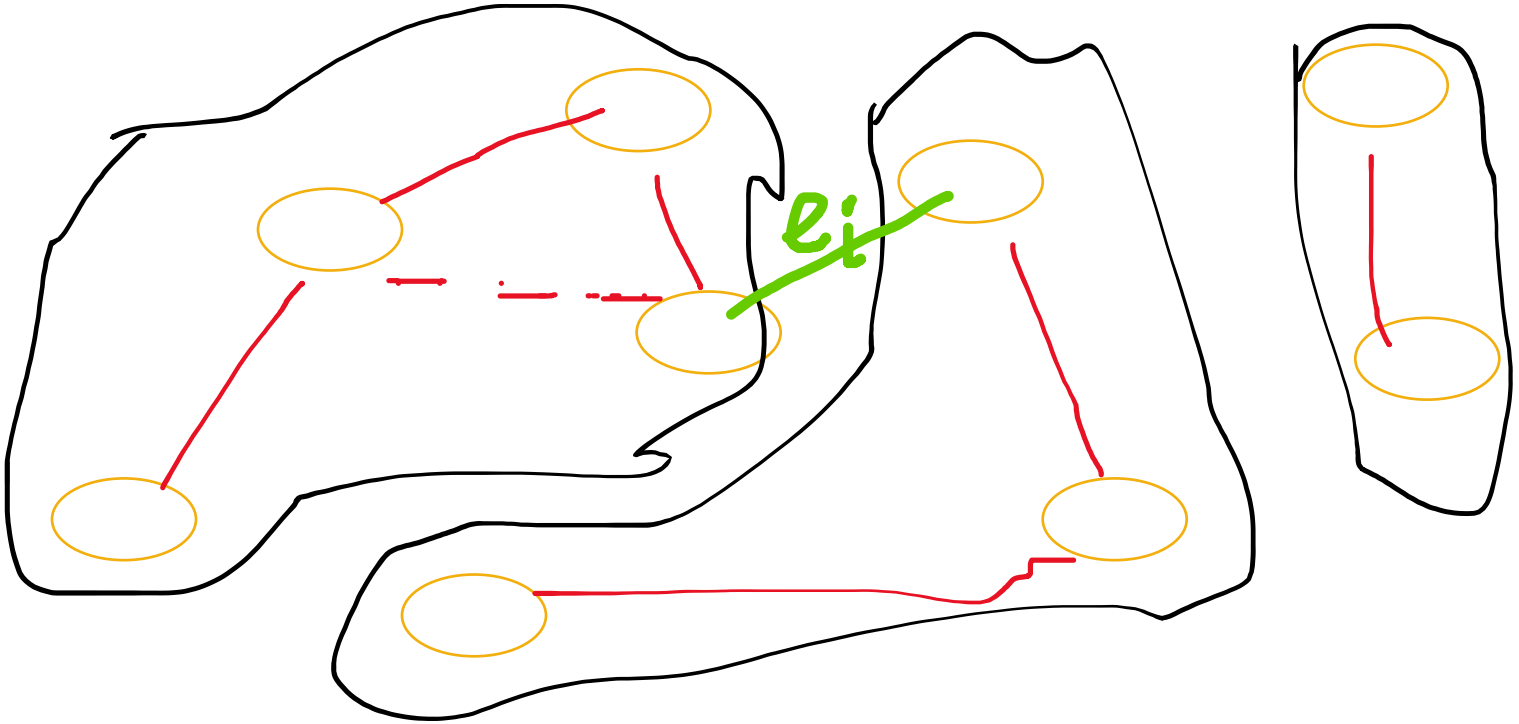
What information do we need?



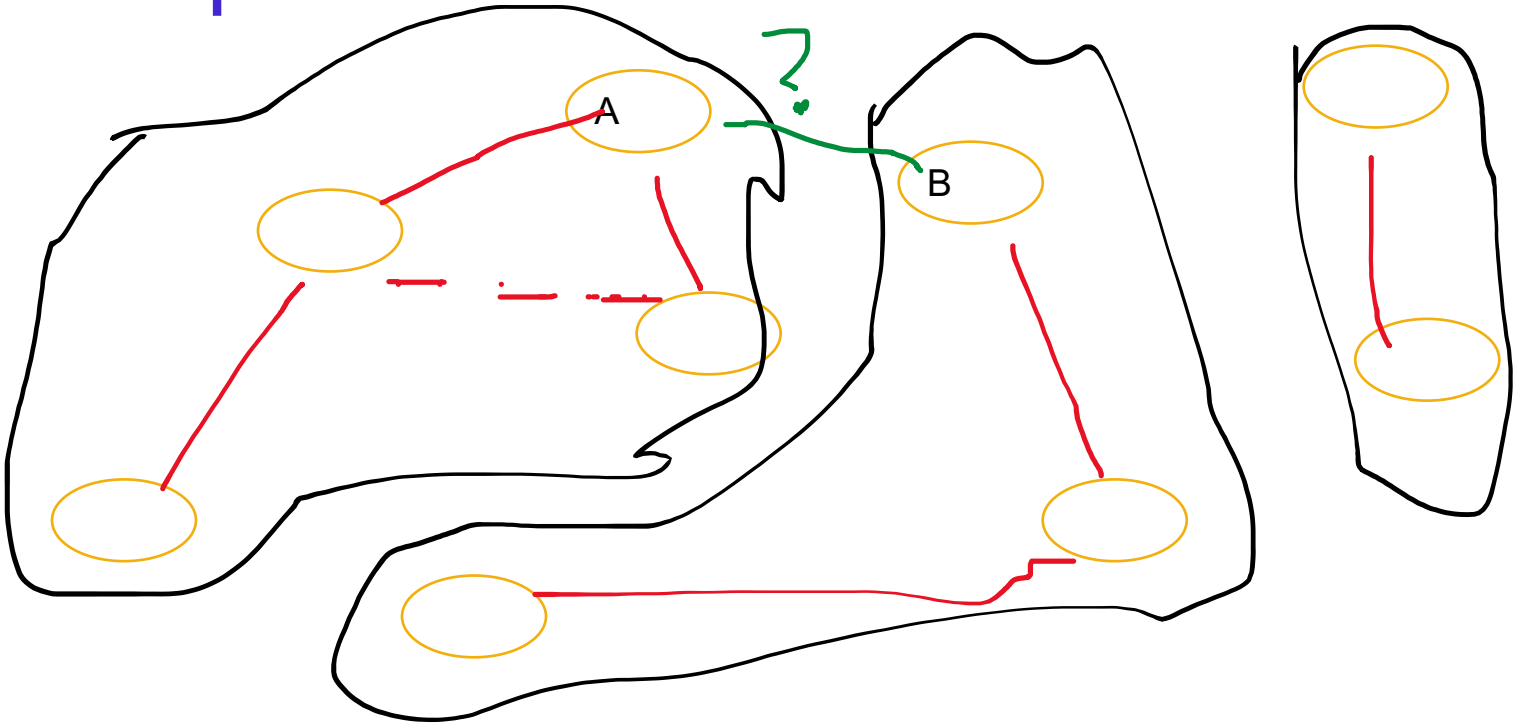
Partition of vertices into components



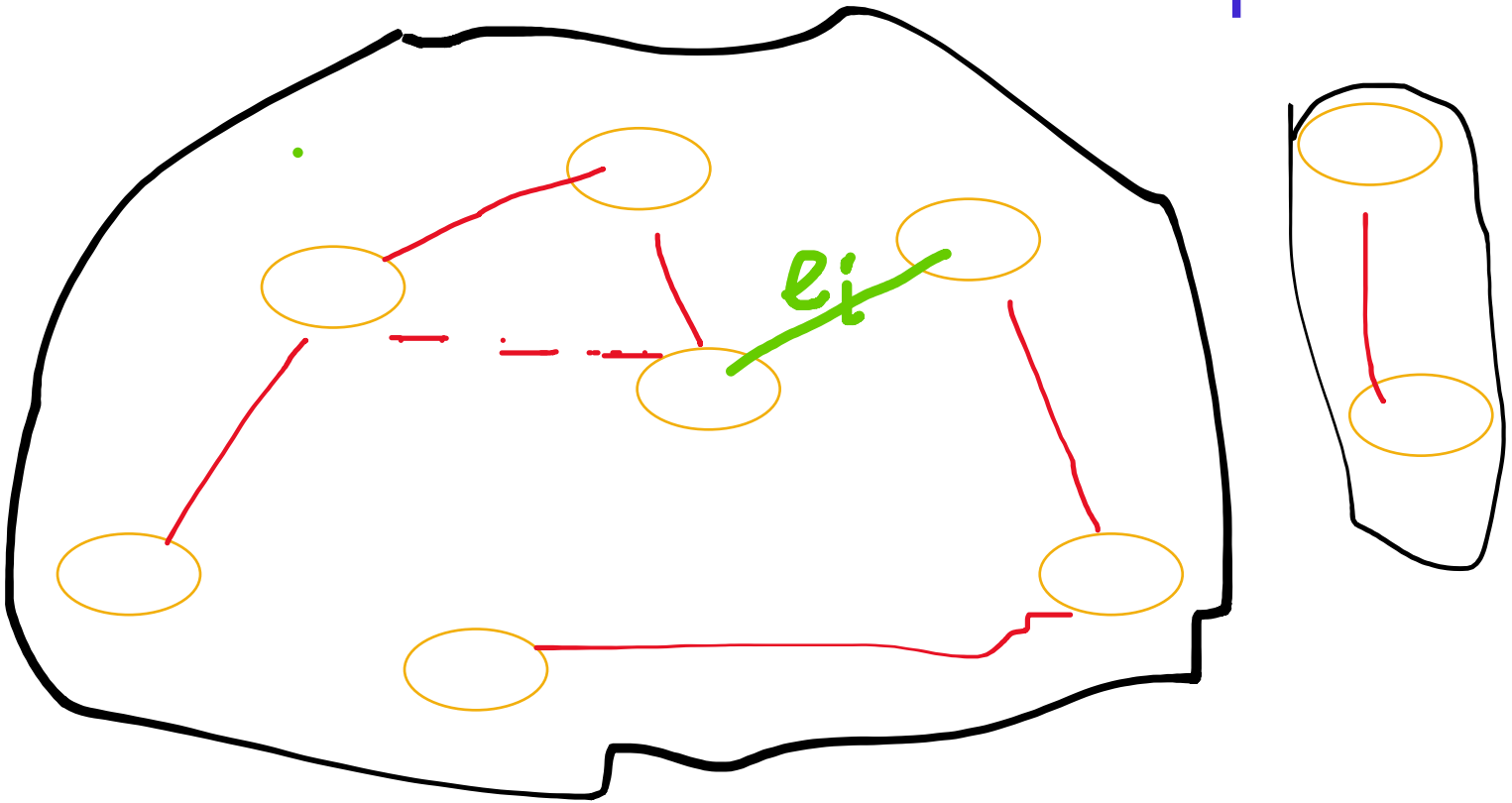
Partition of vertices into components



One step: are two vertices in same component?



Partition of vertices into components



Let's ask our standard DS questions

- What kind of object do we need to keep track on in Kruskal's algorithm? We need to keep track of the way the edges added to the MST divide up the vertices into components.
- What do we need to know in one step? Are two vertices in the same component?
- How does the structure change in one step? If we add an edge, it merges the two components into one.

DSDS matches our requirements

- DSDS stands for Disjoint Sets Data Structure.
- Given a set of objects, DSDS manages partitioning the set into disjoint subsets.
- It does the following operations:
  - Makeset(S): puts each element of S into a set by itself.
  - Find(u): it returns the name of the subset containing u.
  - Union(u,v): it unions the set containing u with the set containing v.

Kruskal's algorithm USING DSDS

- procedure Kruskal(G: graph,w: edge weights)
- Makeset(V), X = { }
- Sort the edges in E in increasing order by weight.
- For all edges (u,v) in E until X is a connected graph
  - $u' = find(u), v' = find(v)$
  - if u' ≠ v':
    - Add edge (u,v) to X
    - Union(u',v'). Note: only leaders merged

Subroutines of Kruskal's

- makeset (u): This creates a set with one element, u
- find(u): finds the set to which u belongs
- union(u,v): merges the sets containing u and v

Runtime of Kruskal's:

$$|V|makeset + 2|E|find + (|V| - 1)union + sort(|E|)$$

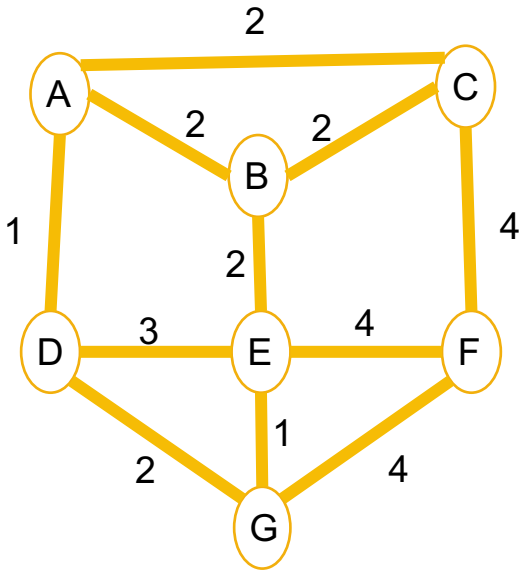
Array implementation of DSDS

Leader[u]: array indexed by vertex, keeps current leader of set containing u

MakeSet(u): Leader[u]=u
Find(u) : return Leader[u]
Merge(u',v'): replace all occurences of v' with u'

Example DSDS version 1 (leader array)

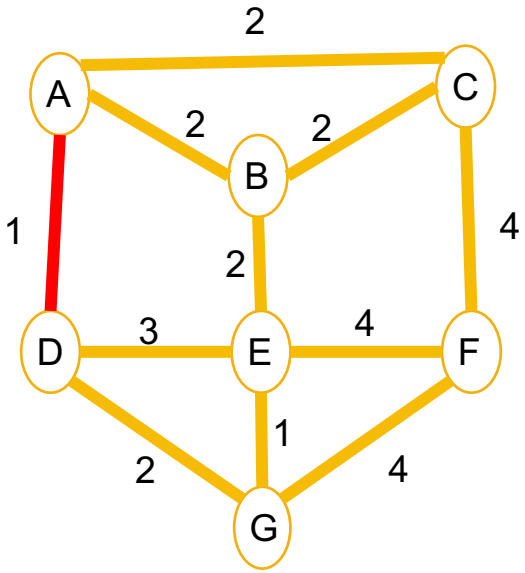
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G

Example DSDS version 1 (leader array)

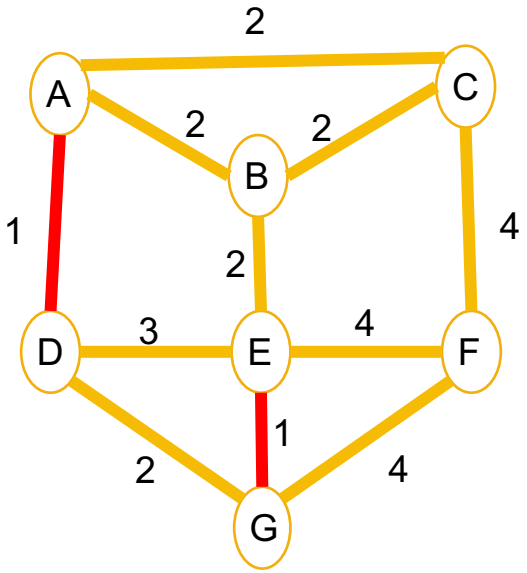
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G

Example DSDS version 1 (leader array)

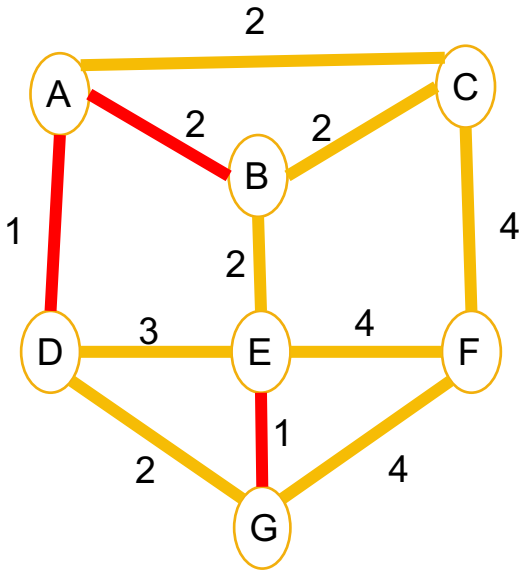
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E

Example DSDS version 1 (leader array)

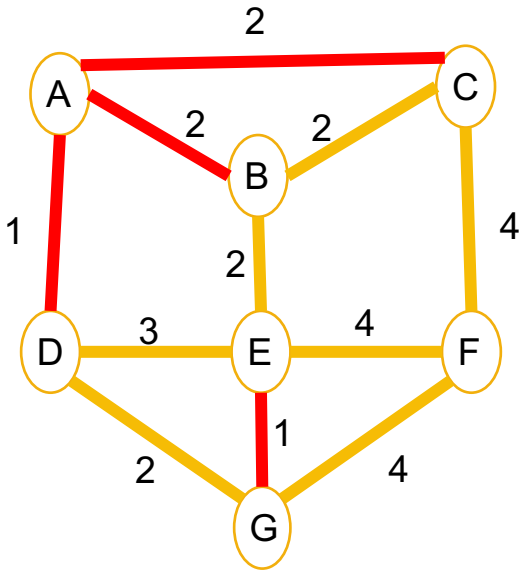
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E

Example DSDS version 1 (leader array)

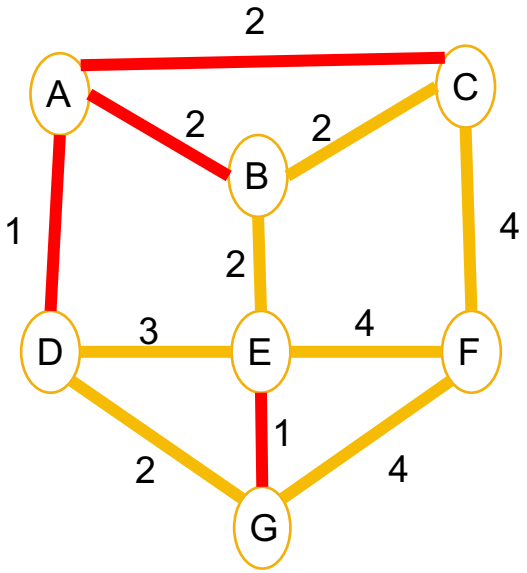
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E
A	A	A	A	E	F	E

Example DSDS version 1 (leader array)

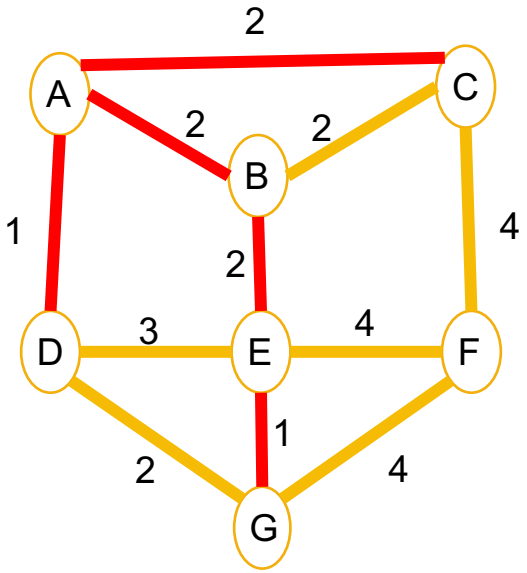
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E
A	A	A	A	E	F	E

Example DSDS version 1 (leader array)

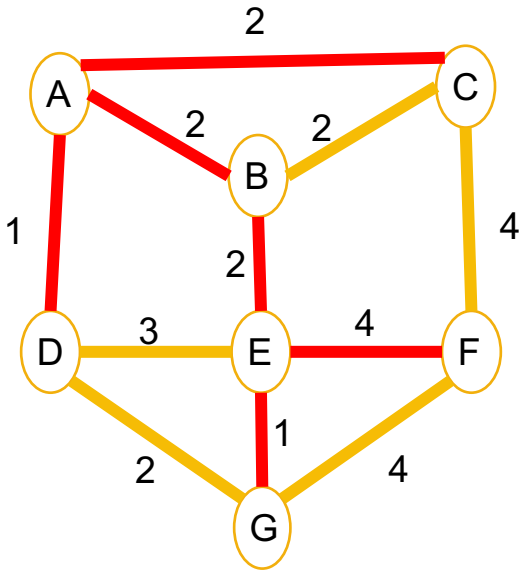
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E
A	A	A	A	E	F	E
A	A	A	A	A	F	A

Example DSDS version 1 (leader array)

(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E
A	A	A	A	E	F	E
A	A	A	A	A	F	A
A	A	A	A	A	A	A

DSDS VERSION 1 (leader array)

- Keep an array Leader(u) indexed by element
- In each array position, keep the leader of its set
- Makeset(u):
- Find(u) :
- union(u,v) :
- Total time:

DSDS VERSION 1

- Keep an array Leader(u) indexed by element
- In each array position, keep the leader of its set
- Makeset(v): Initialize to self, O(1)
- Find(u) : return Leader(u), O(1)
- union(u,v) : For each array position, if it's currently Leader(v), change it to Leader(u). (O(|V|) time)

- Kruskal Total time:
  $|V| * O(1) + 2 * |E| * O(1) + (|V| - 1) * O(|V|) + sort(|E|)$
- = $O(|V|^2 + |E| \log |V|)$

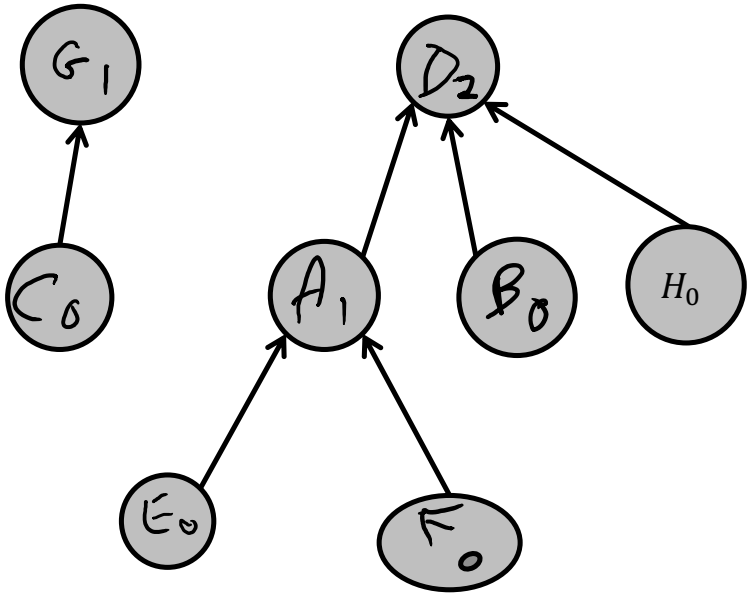
DSDS using trees

- Each set is a rooted tree, with the vertices of the tree labelled with the elements of the set and the root the leader of the set
- Only need to go up to leader, so just need parent pointer

union-by-rank

- vertices of the trees are elements of a set and each vertex points to its parent that eventually points to the root.
- The root points to itself.
- The root is a convenient representation or name of the set containing it and all of its children.
- The actual information in the data structure is stored in two arrays:
  - $\pi(v)$: the parent pointer (roots point to themselves)
  - rank(v): the height of the tree hanging from v. (This is important for efficiency.)

A data structure for Kruskal's (union-by-rank)



rank(A)=1 $\pi(A) = D$
rank(B)=0 $\pi(B) = D$
rank(C)=0 $\pi(C) = G$
rank(D)=2 $\pi(D) = D$
rank(E)=0 $\pi(E) = A$
rank(F)=0 $\pi(F) = A$
rank(G)=1 $\pi(G) = G$
rank(H)=0 $\pi(H) = D$

subroutines.

- procedure makeset(x)
  - $\pi(x) := x$
  - rank(x):=0

Find version 2.1

- procedure find(x)
  - $while\big(x \neq \pi(x)\big)$
    - $x := \pi(x)$
  - return x

subroutines.

- procedure union(x,y)
  - rx:=find(x)
  - ry:=find(y)
  - if rx=ry then return
  - if rank(rx)>rank(ry) then
    - $\pi(ry) := rx$
  - else
    - $\pi(rx) := ry$
    - if rank(rx)=rank(ry) then
      - rank(ry):=rank(rx)+1

subroutines.

- procedure union(x,y)
  - rx:=find(x)
  - ry:=find(y)
  - if rx=ry then return
  - if rank(rx)>rank(ry) then
    - $\pi(ry) := rx$
  - else
    - $\pi(rx) := ry$
    - if rank(rx)=rank(ry) then
      - rank(ry):=rank(rx)+1

- To save on runtime, we must keep the heights of the trees short.
- So union of two ranks points the smaller rank to the bigger rank, that way, the tree will stay the same height.
- If the ranks are equal, then it increments one rank and points the smaller to the bigger. (this is the only way a rank can increase.)

subroutines.

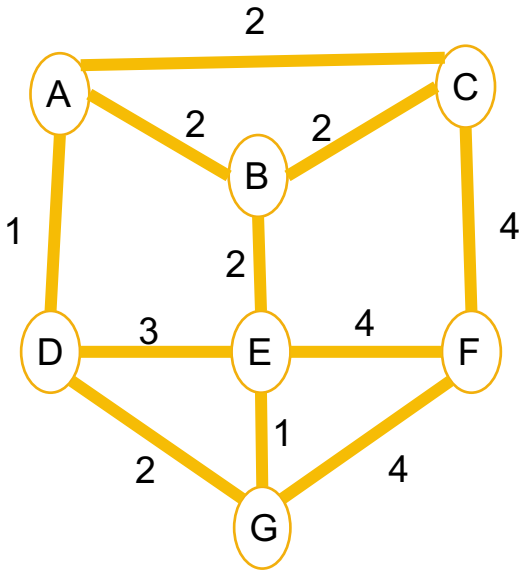
- procedure makeset(x)
  - $\pi(x) := x$
  - rank(x):=0
- procedure find(x)
  - $while\big(x \neq \pi(x)\big)$
    - $x := \pi(x)$
  - return x

makeset=O(1)
find=O(height of tree containing x)
union=O(find)

- procedure union(x,y)
  - rx:=find(x)
  - ry:=find(y)
  - if rx=ry then return
  - if rank(rx)>rank(ry) then
    - $\pi(ry) := rx$
  - else
    - $\pi(rx) := ry$
    - if rank(rx)=rank(ry) then
      - rank(ry):=rank(rx)+1

Example DSDS version 1 (tree)

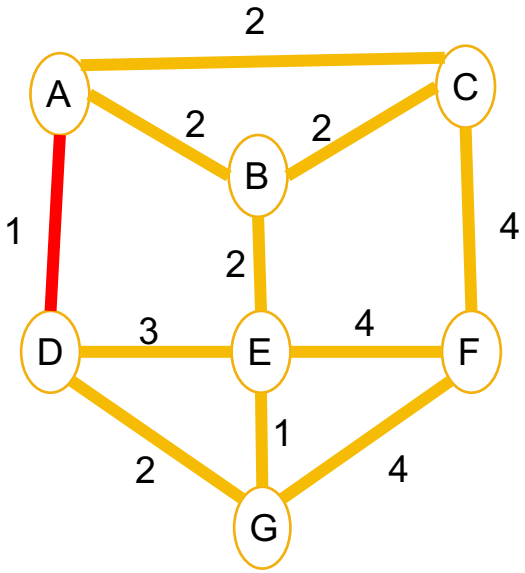
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A,0	B,0	C,0	D,0	E,0	F,0	G,0

Example DSDS version 2 (tree)

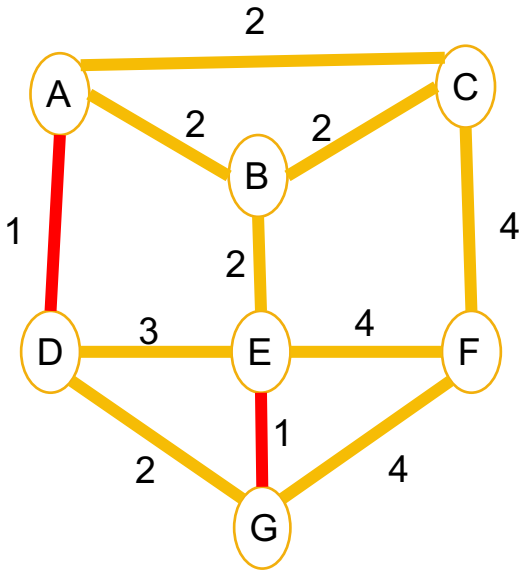
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A, 1	B,0	C,0	A,0	E,0	F,0	G,0

Example DSDS version 2 (tree)

(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4

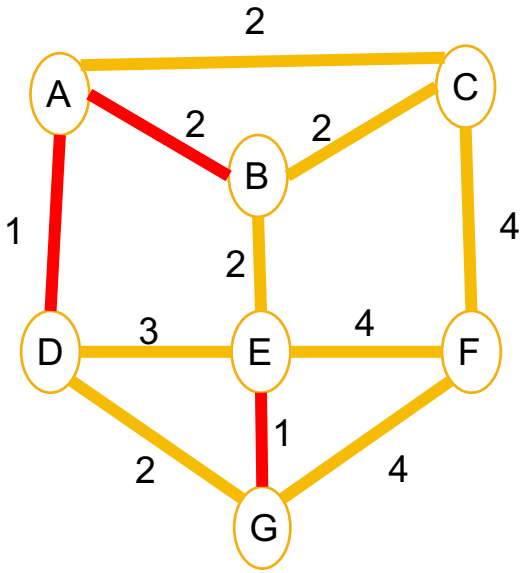


A	B	C	D	E	F	G
A	B	C	A	E	F	G
A, 1	B	C	A	E, 1	F	E

Default: no rank given means rank is 0

Example DSDS version 1 (tree)

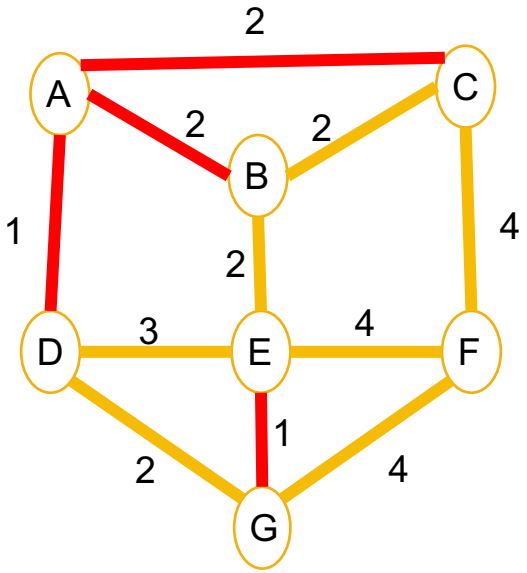
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A,1	A	C	A	E, 1	F	E

Example DSDS version 1 (tree)

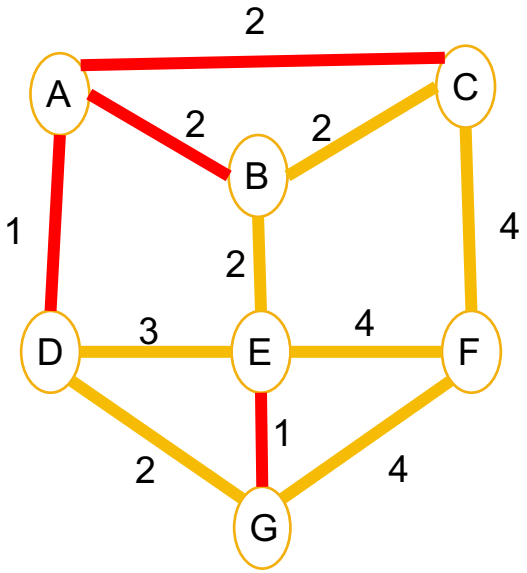
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E
A, 1	A	A	A	E, 1	F	E

Example DSDS version 2 (tree)

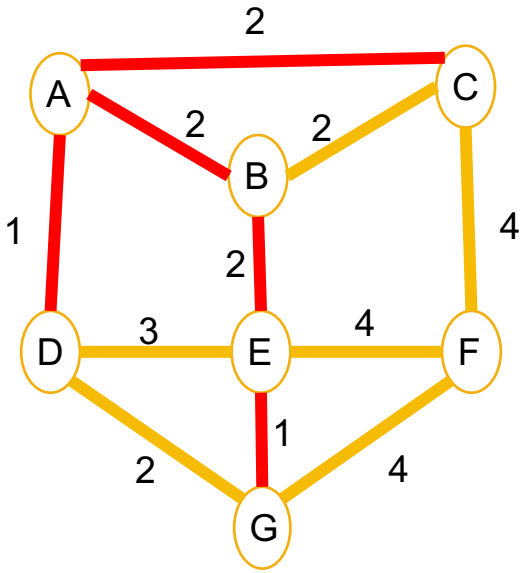
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E
A,1	A	A	A	E, 1	F	E

Example DSDS version 2 (tree)

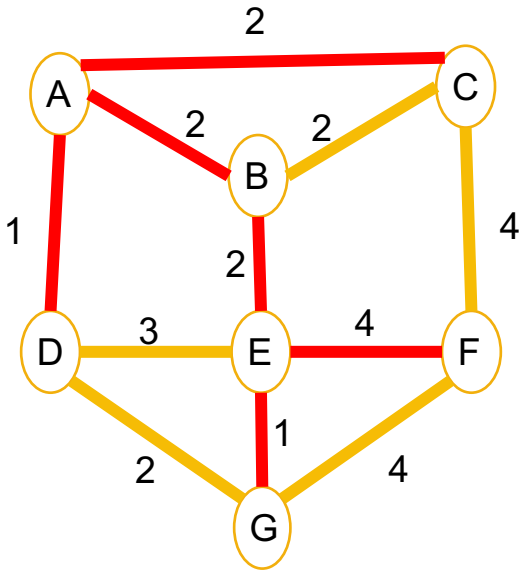
(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E
A	A	A	A	E, 1	F	E
A,2	A	A	A	A, 1	F	E

Example DSDS version 2 (tree)

(A,D)=1
(E,G)=1
(A,B)=2
(A,C)=2
(B,C)=2
(B,E)=2
(D,G)=2
(D,E)=3
(E,F)=4
(F,G)=4



A	B	C	D	E	F	G
A	B	C	A	E	F	G
A	B	C	A	E	F	E
A	A	C	A	E	F	E
A	A	A	A	E,1	F	E
A	A	A	A	A, 1	F	E
A, 2	A	A	A	A, 1	A	E

Ranks correspond to heights of trees.

- If a vertex has rank k, then the height of its tree is k.
- Proof:
- Base Case: a vertex by itself has rank 0.
- Suppose that every vertex of rank k has a tree of height k.
- Consider a vertex v of rank k+1. Then it must have been a vertex of rank k at some point. It increased when another vertex u of rank k was unioned with v. The parent of u is v so the edge (u,v) along with the height of the tree hanging from u makes a tree of height k+1 hanging from v.

height of tree

- any root node of rank k has at least 2^k vertices in its tree.

proof:

Base Case: a root of rank 0 has 1 vertex

suppose a root of rank k has at least 2^k vertices in its tree

Then a root of rank k+1 can only be made by unioning 2 roots each of rank k so a root of rank k+1 must have at least $2^k + 2^k = 2^{k+1}$vertices in its tree.

Max height of tree

- Suppose there are n vertices.
- A vertex of rank log(n) has at least n vertices in its tree.
- A vertex of any higher rank will have more than n vertices in its tree.
- Therefore the maximum height of a tree with n vertices is log(n).

Runtime

makeset=O(1)
find=O(height of tree containing x)
union=O(find)

- makeset (u): This creates a set with one element, u
- find(u): finds the set to which u belongs
- union(u,v): merges the sets containing u and v

Runtime of Kruskal's:

$$|V|makeset + 2|E|find + (|V| - 1)union + sort(|E|)$$

Runtime

makeset=O(1)
find=O(log(n))
union=O(log(n))

- makeset (u): This creates a set with one element, u
- find(u): finds the set to which u belongs
- union(u,v): merges the sets containing u and v

Runtime of Kruskal's:

$$|V|makeset + 2|E|find + (|V| - 1)union + sort(|E|)$$
$$|V|O(1) + 2|E|O(\log(|V|)) + (|V| - 1)O(\log(|V|)) + |E|\log(|E|)$$
$$O(|V| + |E|\log(|V|) + |V|\log(|V|) + |E|\log(|E|))$$
$$O(|E|\log(|V|))$$

Runtime

makeset=O(1)
find=O(log(n))
union=O(log(n))

Runtime of Kruskal's:

$$O(|E|\log(|V|))$$

- Can we improve the runtime of find and union?

- Is it worth it? (will it actually improve the runtime of Kruskal's?)

Coming up:

Quickly growing and slowly growing functions
Amortized analysis
Path compression
Amortized analysis of DSDS